DEFENSIVE SECURITY
TECHNICAL PORTFOLIO

Enterprise Security Infrastructure & SOC Monitoring

BLUE TEAM IMPLEMENTATION REPORT

Environment: VMware virtual lab, Windows Server and Red Hat Enterprise Linux
Scope: Palo Alto NGFW | Splunk Enterprise | Wazuh

Project attribution: This is the Blue Team portion of a collaborative lab project. Windows/Linux administration and offensive-security work by other team members are not included.

Evidence: Screenshots were selected from the original report. Images displaying private paths, API credentials or chat identifiers were excluded; lab hostnames and private IP addresses may remain.

MODULES

01 Palo Alto NGFW / Network Segmentation and Policy

02 Splunk Enterprise / Centralised Security Monitoring

03 Wazuh / Endpoint Detection and Active Response

Palo Alto NGFW: Network Segmentation, Access Control and Data Filtering

Project: Enterprise Security Infrastructure and SOC Monitoring

Area: Blue Team / Network Defence

Environment: VMware training lab with Windows and Red Hat Enterprise Linux endpoints

Objective and environment

The first defensive task was to establish a centrally managed network perimeter and separate systems according to their purpose. A Palo Alto Networks next-generation firewall (NGFW) provided inter-zone routing, access control, outbound NAT and policy-based inspection. The environment consisted of four distinct zones:

Zone	Lab subnet/interface	Purpose
DMZ	192.168.10.0/24	Externally accessible web services
Internal	192.168.20.0/24	Internal Windows and Linux systems
Management	192.168.30.0/24	Administrative workstation and security monitoring systems
Untrust	External uplink (10.10.10.2 in the lab)	Outbound connectivity

The purpose of this design was not to make every zone universally reachable. Administrative access, internal services and Internet access were handled through separate rules. The addresses above belong to the isolated laboratory environment.

1. Initial firewall deployment and Layer 3 interfaces

1. Attached the required virtual interfaces in VMware, accessed the management console and established an initial management connection.
2. Set the firewall hostname and time zone, then changed the management address to a stable configuration.
3. Created the DMZ_Zone, Internal_Zone, Mgmt_Zone and Untrust_Zone security zones.
4. Assigned the corresponding Ethernet interfaces to Layer 3 mode, associated each interface with its security zone and provided the interface gateway address.
5. Associated the Layer 3 interfaces with the default virtual router, committed the changes and checked interface status.
6. Applied restricted interface management profiles to permit necessary diagnostic traffic without exposing administrative services unnecessarily.

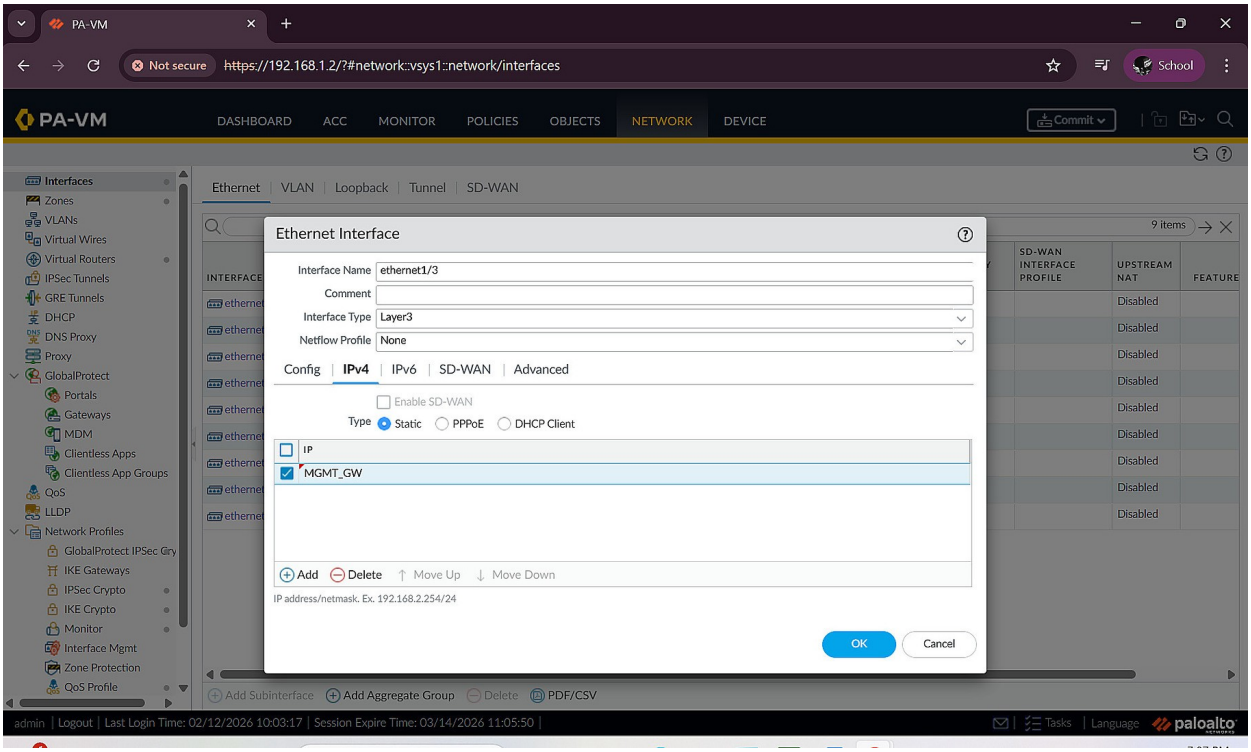


Figure 1. Palo Alto Layer 3 interface settings in the lab.

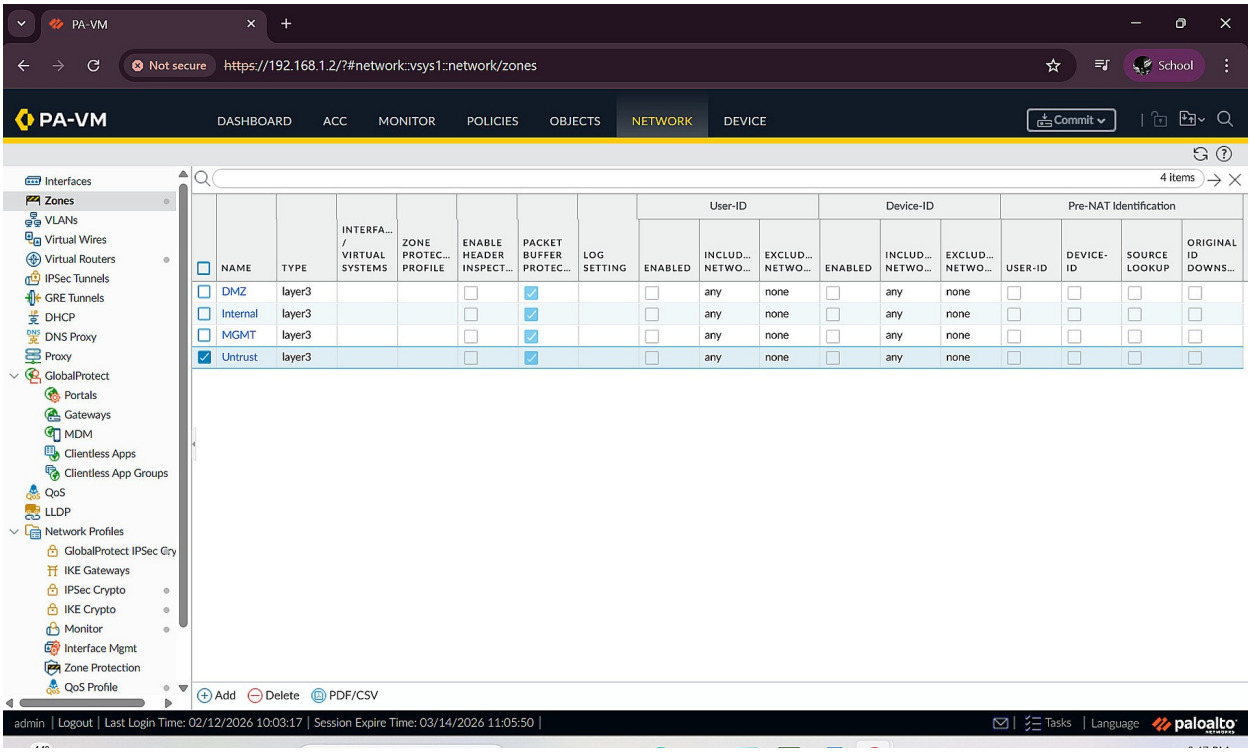


Figure 2. Interface status and zone assignment overview.

2. Routing, Source NAT and Internet access

After the interface configuration, the firewall required explicit routing and policy controls:

7. Added the DMZ, Internal, Management and Untrust interfaces to the virtual router.
8. Configured a default route, 0.0.0.0/0, through the Untrust side and its next-hop gateway.
9. Created an outbound Source NAT policy using Dynamic IP and Port (DIPP), translating permitted private clients to the external interface address.
10. Added the related allow rule under **Policies > Security**. The rule defined source zones, destination zone, allowed traffic and logging at session end.

11. Tested connectivity from an administrative Windows workstation using gateway pings and external DNS/IP connectivity checks.

Source NAT alone does not authorise traffic in this configuration. The corresponding security policy must also allow the session. I verified this distinction while troubleshooting the initial traffic flow.

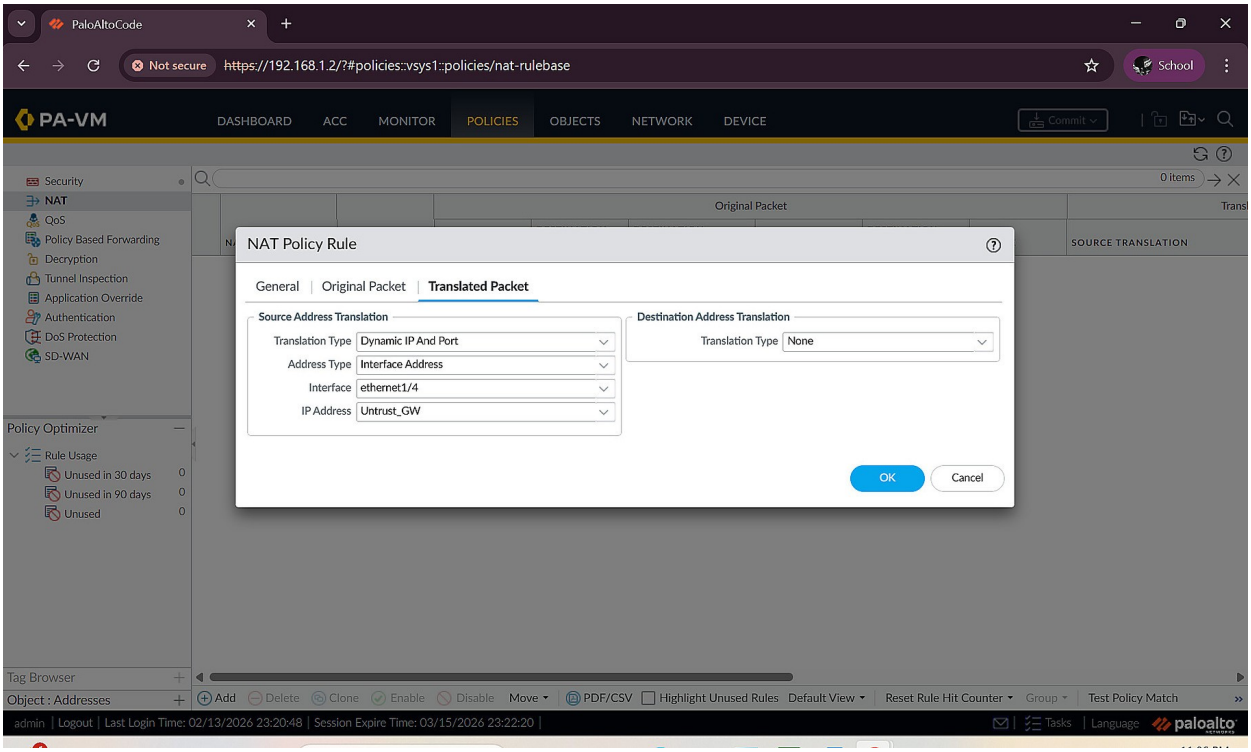


Figure 3. Outbound NAT rule configuration.

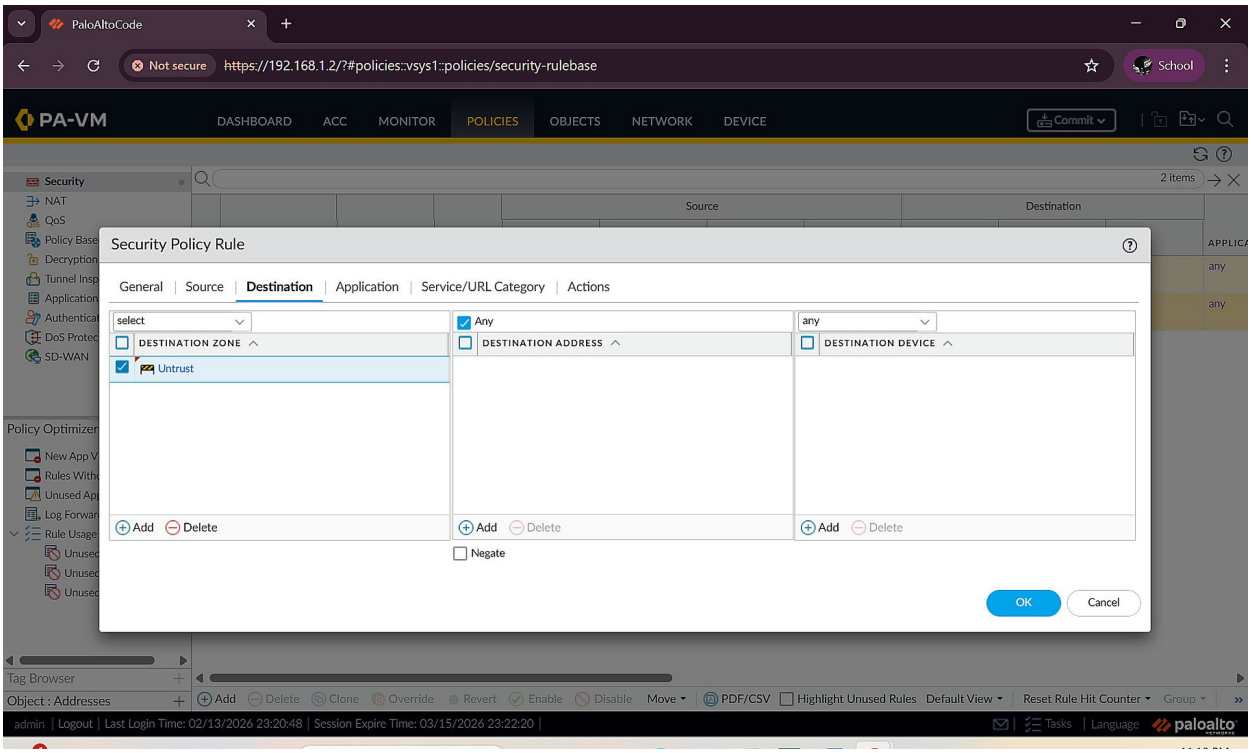


Figure 4. Security rule with explicit source and destination zones.

3. Protecting firewall management access

A dedicated management zone was used to separate administrative access from ordinary traffic. I created an interface management profile allowing **HTTPS and SSH** for administration, together with limited ICMP for connectivity tests. This profile was bound to the management-facing Layer 3 interface rather than applied

indiscriminately to every zone. Internal and DMZ interfaces retained more restricted diagnostic settings; the Untrust side was not intended to expose these services.

The configuration was committed and checked from the designated administrative workstation. This demonstrated how interface management profiles and zone-based security policies serve different purposes: one determines which services the firewall interface itself exposes; the other controls transit traffic between networks.

4. Inter-zone policy and segmentation tests

The lab requirements included management access to designated hosts and restrictions on less-trusted systems reaching sensitive internal resources. The Admin_to_All_Zones rule was created with the Management zone as the source and specified Internal/DMZ destinations. Session-end logging was enabled to support later investigation.

During testing, one Linux host sent packets through an unintended gateway because of a host networking/ARP issue. The network adapter settings were corrected, the stale ARP entry was cleared and connectivity was retested. Administrative reachability to the intended hosts was then checked without opening blanket access from the DMZ to the Internal network.

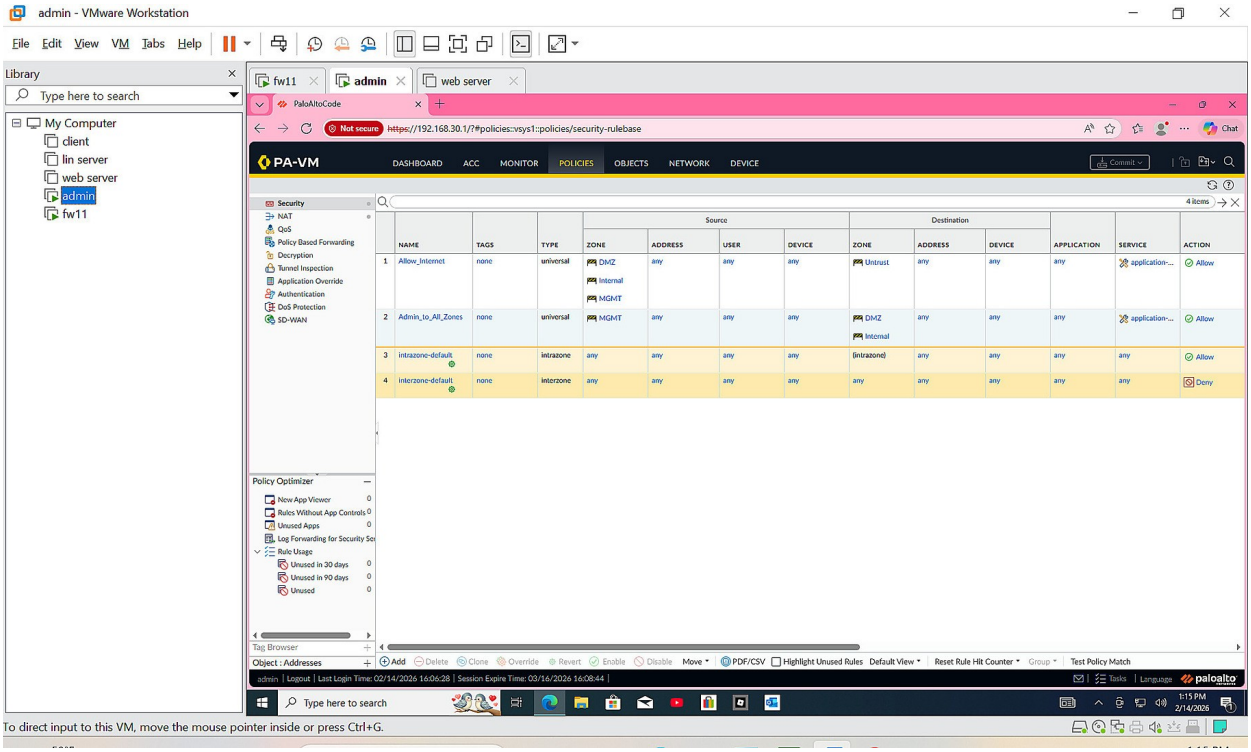


Figure 5. Policy list used for segmented access and management traffic.

5. URL filtering

The next task was to enforce restrictions on outbound web content. Under **Objects > Security Profiles > URL Filtering**, I created a profile for categories required by the training scenario. In addition to adult content and gambling, the documented policy included higher-risk categories such as malware, phishing, command-and-control, proxy avoidance and certain file-sharing services.

The profile was attached to the appropriate Internet access security rule, and the changes were committed. Testing from an internal Linux endpoint used both a browser and command-line requests. The firewall's blocked-page response and traffic records were used as evidence that the profile was being applied.

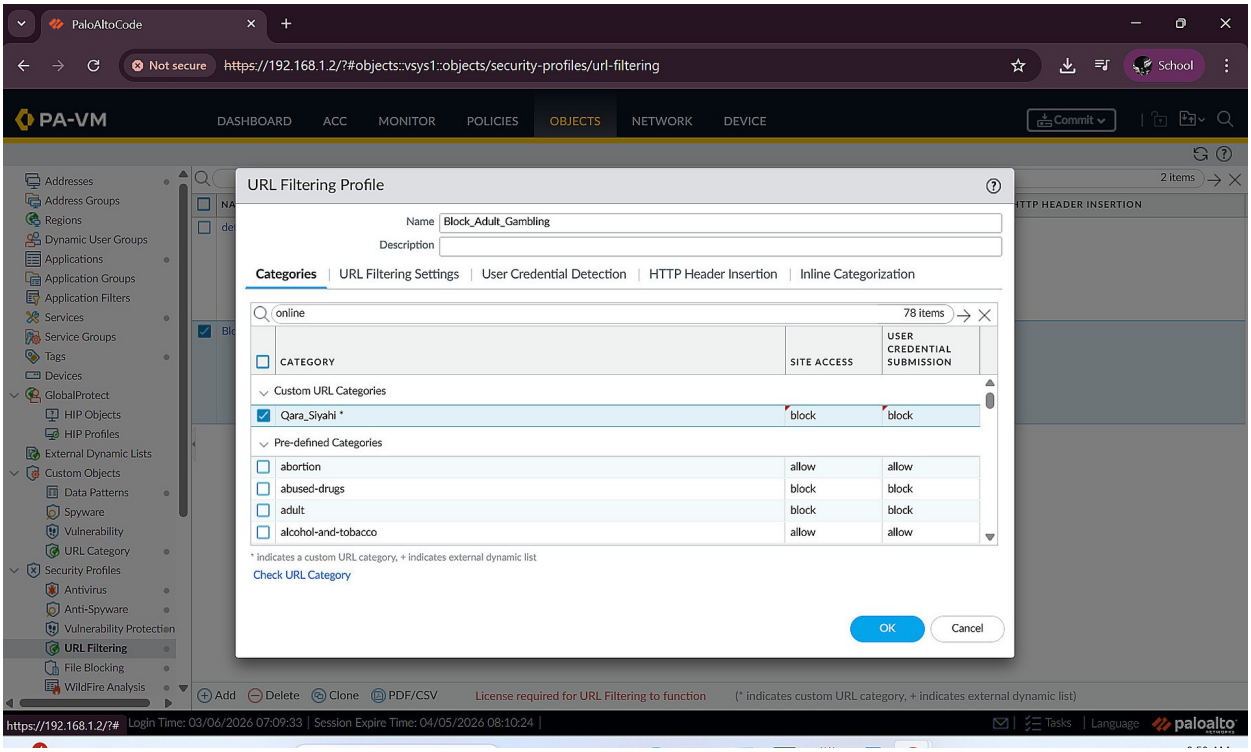


Figure 6. Selected URL categories and their enforcement actions.

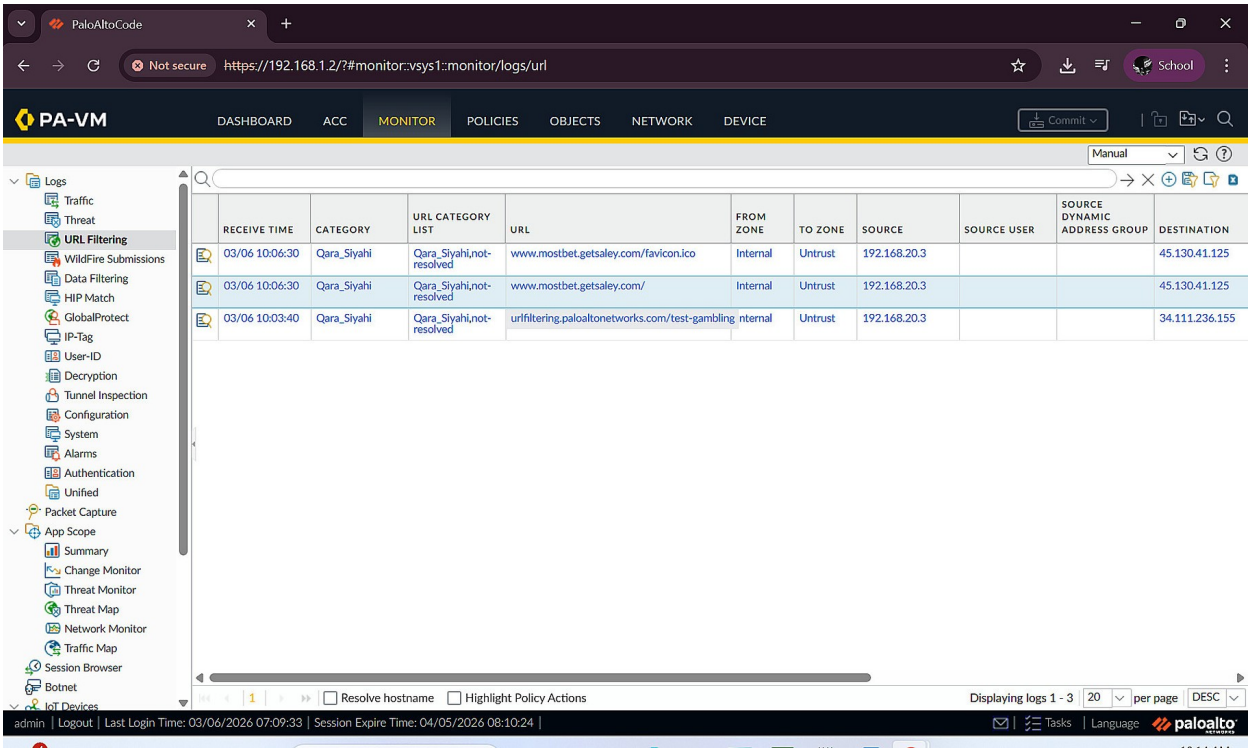


Figure 7. URL filtering activity in the firewall traffic logs.

6. Data filtering / DLP proof of concept

To test an outbound data-loss-prevention control, I configured custom data patterns for a laboratory-only confidential keyword and a 16-digit numeric pattern representing test payment-card-like values. These patterns were included in a Data Filtering profile with an **Upload** direction, a block threshold and high-severity event logging.

The profile was attached to the outbound security rule. From an internal endpoint, test values were submitted to a public DLP testing page. The firewall response and related log entries were checked to confirm that the selected test traffic was blocked.

This was a **proof of concept**, not a production payment-data classifier. A simple 16-digit pattern can also match unrelated numbers and should not be treated as reliable card verification. It was sufficient for the stated training exercise.

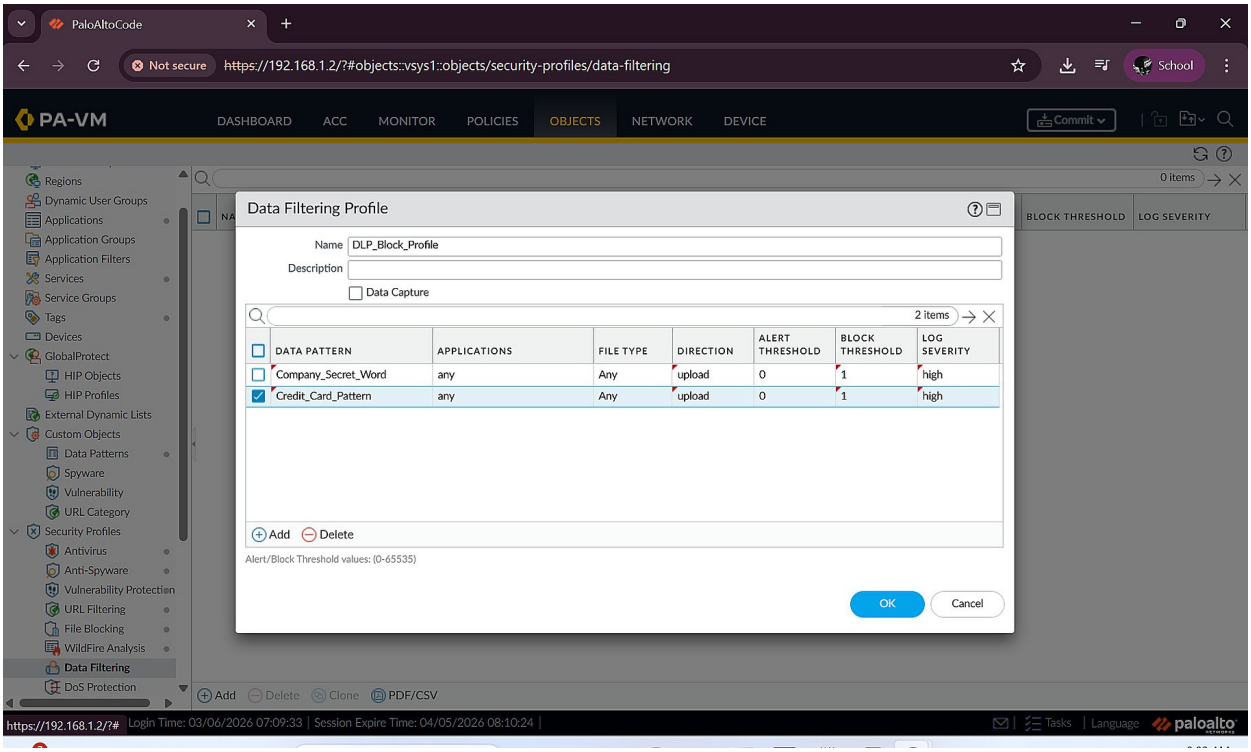


Figure 8. Data Filtering profile and laboratory matching rules.

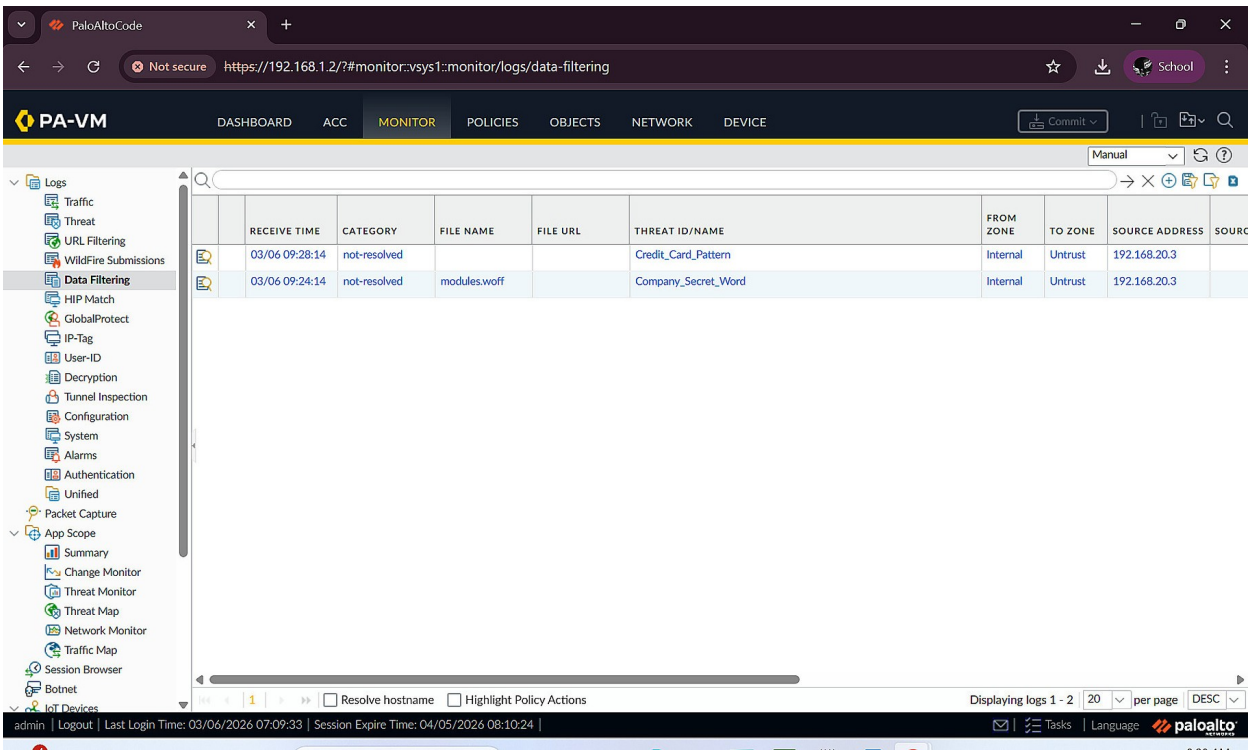


Figure 9. Logged Data Filtering events after testing.

Results and lessons learned

The lab demonstrated functional separation of security zones, controlled administration, outbound NAT, URL filtering and data filtering. I validated the changes with connectivity tests, policy checks and firewall logs. The most important lessons were to distinguish routing/NAT from authorisation, commit and verify each policy change, minimise administrative exposure and use logs to validate what the firewall actually enforced.

Splunk Enterprise: Centralised Logging, SSH Investigation and SOC Dashboards

Project: Enterprise Security Infrastructure and SOC Monitoring

Area: Blue Team / Security Monitoring and Alerting

Environment: Splunk Enterprise on Red Hat Enterprise Linux, Windows and Linux log sources

Objective

The SIEM task focused on collecting logs from multiple endpoints in one place, identifying authentication-related indicators and presenting findings through dashboards and alerts. Splunk Enterprise was placed in the Management zone. Linux and Windows endpoints in other zones used Splunk Universal Forwarders to send their data over approved network paths.

1. Deploying Splunk Enterprise on RHEL

12. Prepared a dedicated RHEL virtual machine and installed the Splunk Enterprise RPM package.
13. Launched the Splunk service and accessed its web management interface from the permitted network.
14. Configured startup behaviour so Splunk would be available after system reboot.
15. Configured the Linux host firewall for the necessary service access while avoiding unnecessary open ports.
16. Enabled a receiving endpoint in Splunk for Universal Forwarder traffic on **TCP 9997**.

The deployment separated the web interface used for analysis from the ingestion path used by endpoint agents.

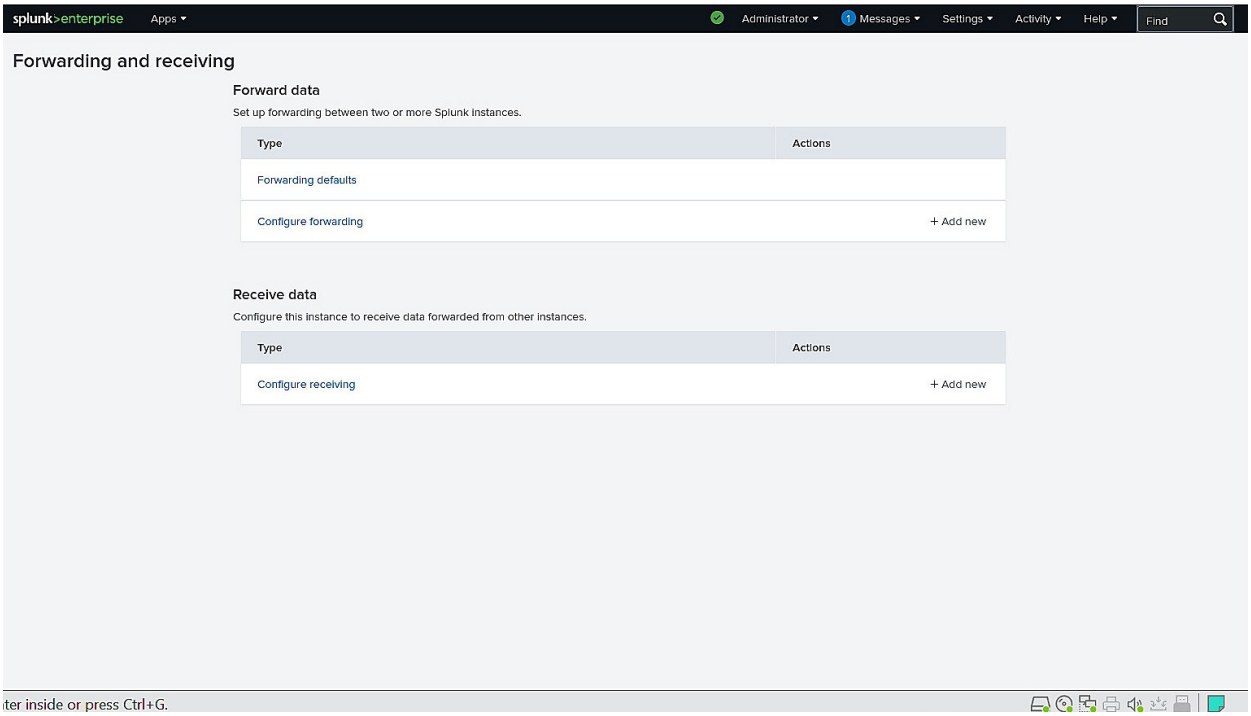


Figure 1. Splunk receiving/forwarding settings in the training environment.

2. Restricting log-forwarding traffic through Palo Alto

Because the Splunk server was hosted in the Management zone, sending logs to it required an explicit firewall rule. A service object was created for the Splunk forwarding port (9997/tcp), followed by a rule that allowed the approved source zones to connect to the designated Splunk server. Session logging was enabled for observability.

The goal was to allow log ingestion without giving endpoint networks unrestricted access to the monitoring infrastructure. This was tested by checking endpoint connectivity and the appearance of received events in Splunk.

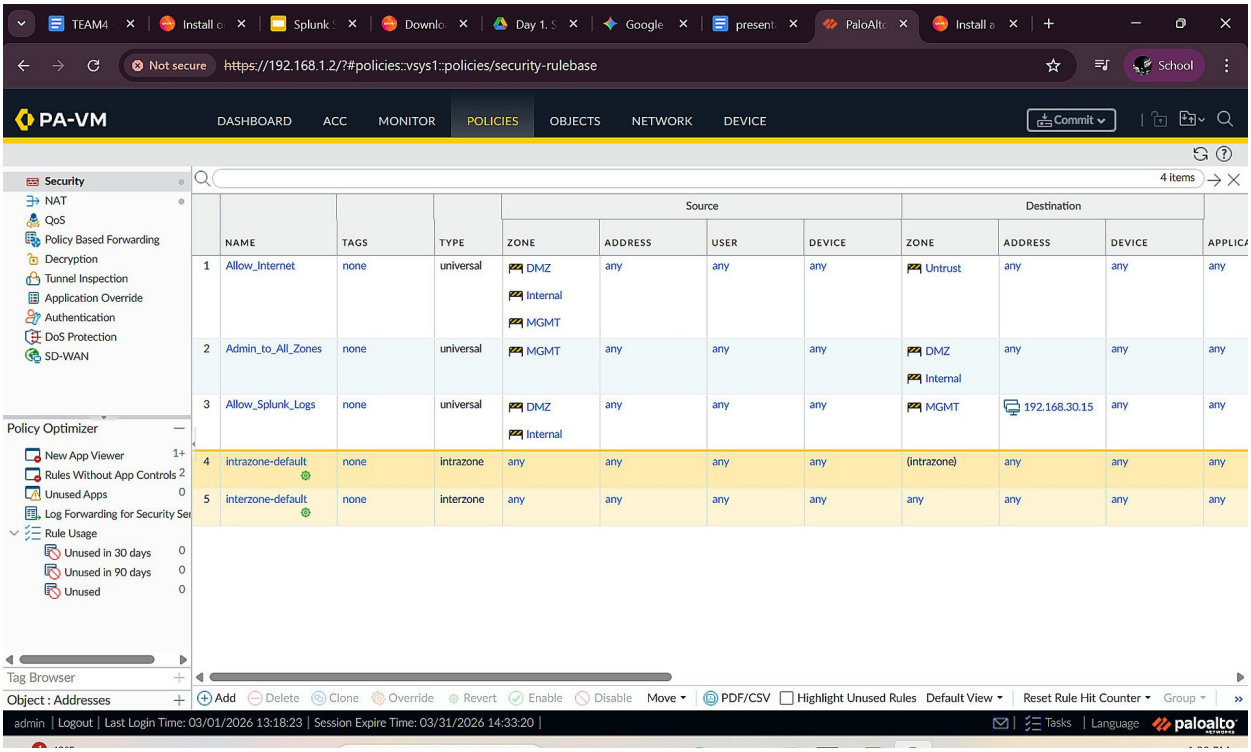


Figure 2. NGFW policies limiting cross-zone access to the monitoring server.

3. Forwarding logs from Linux hosts

Splunk Universal Forwarder was deployed on the Linux systems that supplied operational and security telemetry. For each endpoint, the process included installing the forwarder, identifying the central Splunk receiver, configuring the monitored sources and starting the forwarder service.

A representative command used during the exercise was:

./splunk add forward-server <SPLUNK_SERVER_IP>:9997
Host-specific addresses are deliberately omitted from reusable examples in the public report. I checked receiving status in the Splunk interface and confirmed that events from multiple lab systems could be searched centrally.

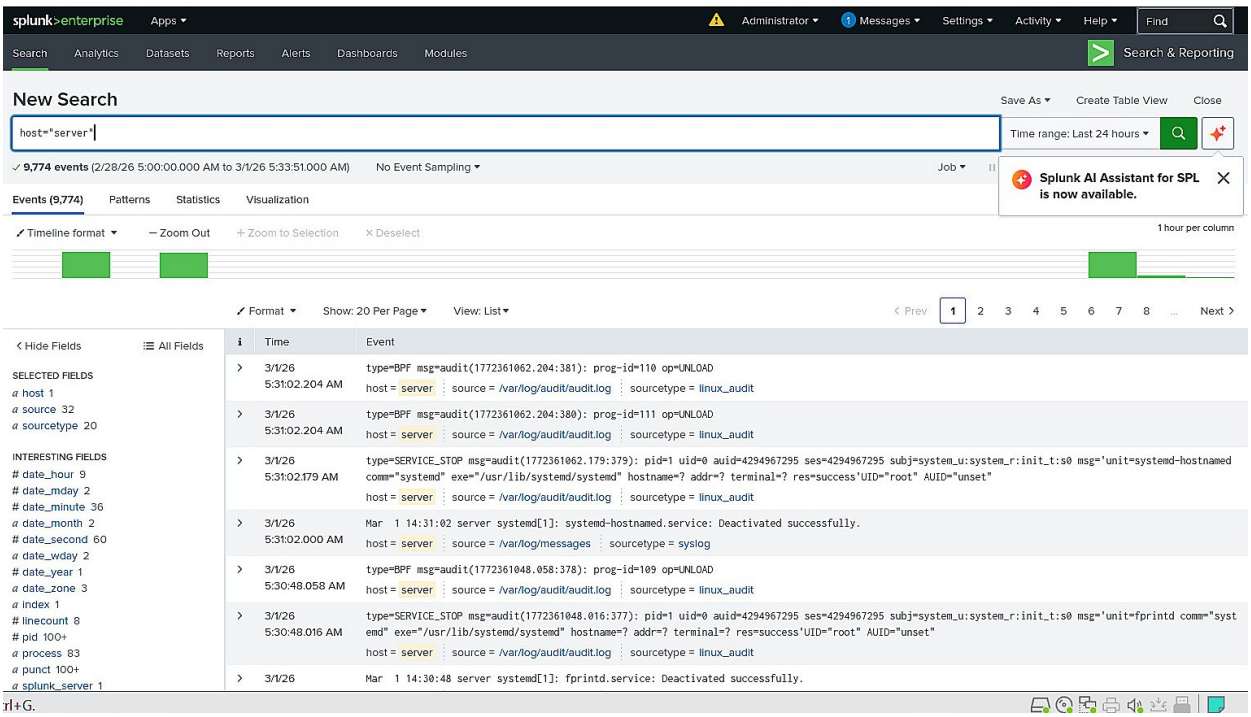


Figure 3. Centralised events collected through the forwarder pipeline.

4. Windows event collection

A Splunk Universal Forwarder was also installed on a Windows administrative endpoint. The setup involved selecting Windows log sources, entering the approved receiving destination and verifying connectivity. The resulting events were checked in Splunk alongside Linux records.

This was useful because the same investigation interface could be used across operating systems, without treating Windows and Linux logs as having identical field names or event structures.

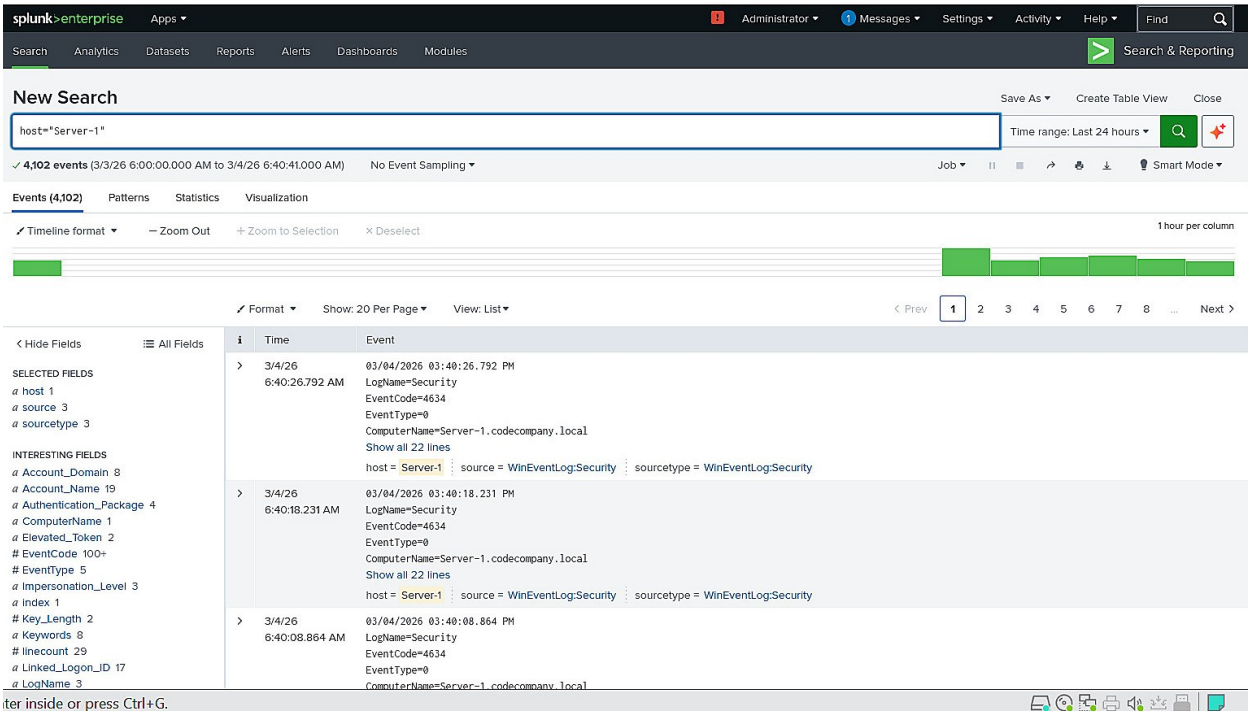


Figure 4. Windows-related events visible in Splunk search results.

5. Investigating failed SSH authentication attempts

The main analysis example was suspicious SSH authentication activity. I searched the collected Linux logs for repeated failed login events, reviewed raw message formats and identified fields useful for investigation, including the source address and targeted account.

The workflow included:

17. Filtering events by the authentication-failure pattern and a relevant time range.
18. Inspecting raw events to understand the source data rather than assuming a predefined schema.
19. Using Splunk's regular-expression field extraction to separate the source IP address and attempted username.
20. Checking the extracted fields against raw messages for accuracy.
21. Aggregating results by source address, username and number of attempts to identify repeated activity.

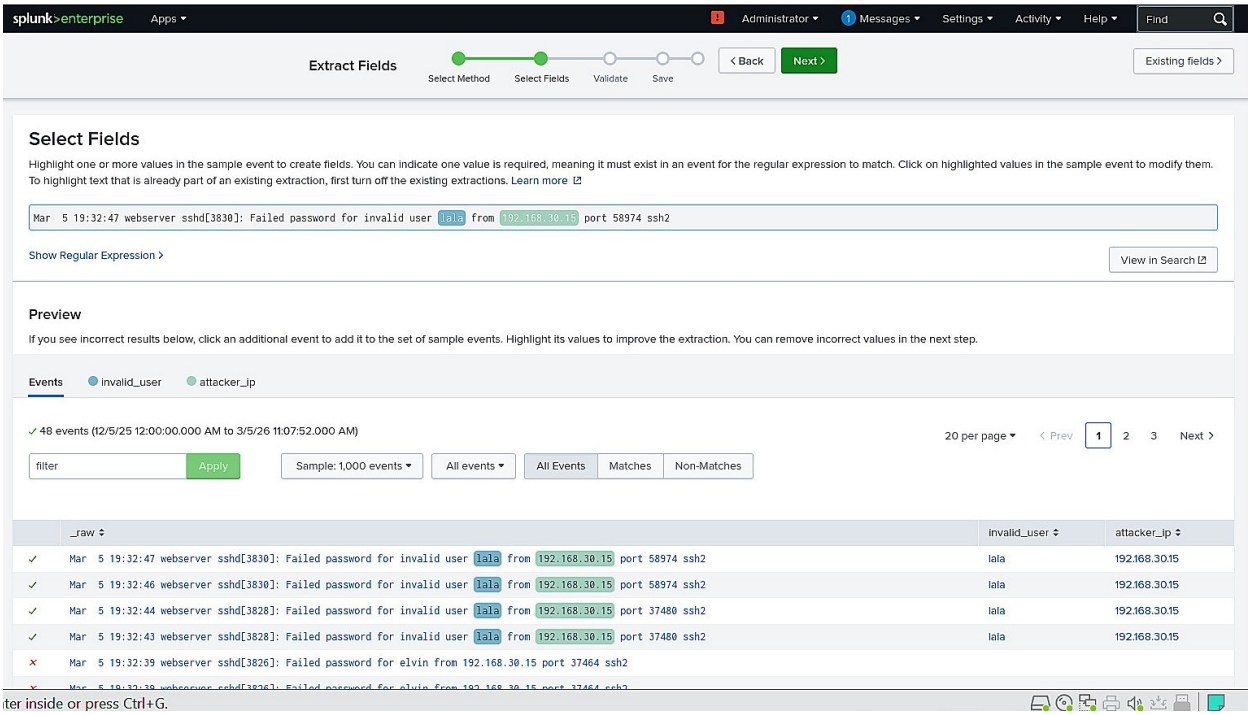


Figure 5. Creating structured fields from authentication logs.

6. Building a SOC dashboard

Based on the extracted fields, I built a dashboard to make authentication activity easier to review. The report documents three visualization styles: a table with conditional formatting, a pie chart showing the distribution of targeted accounts and a single-value indicator representing a summary count.

The table helped identify noisy sources; the chart made account targeting patterns easier to compare; the single-value panel provided a quick overview. During the project, I checked that the dashboard components updated when the search results or time range changed.

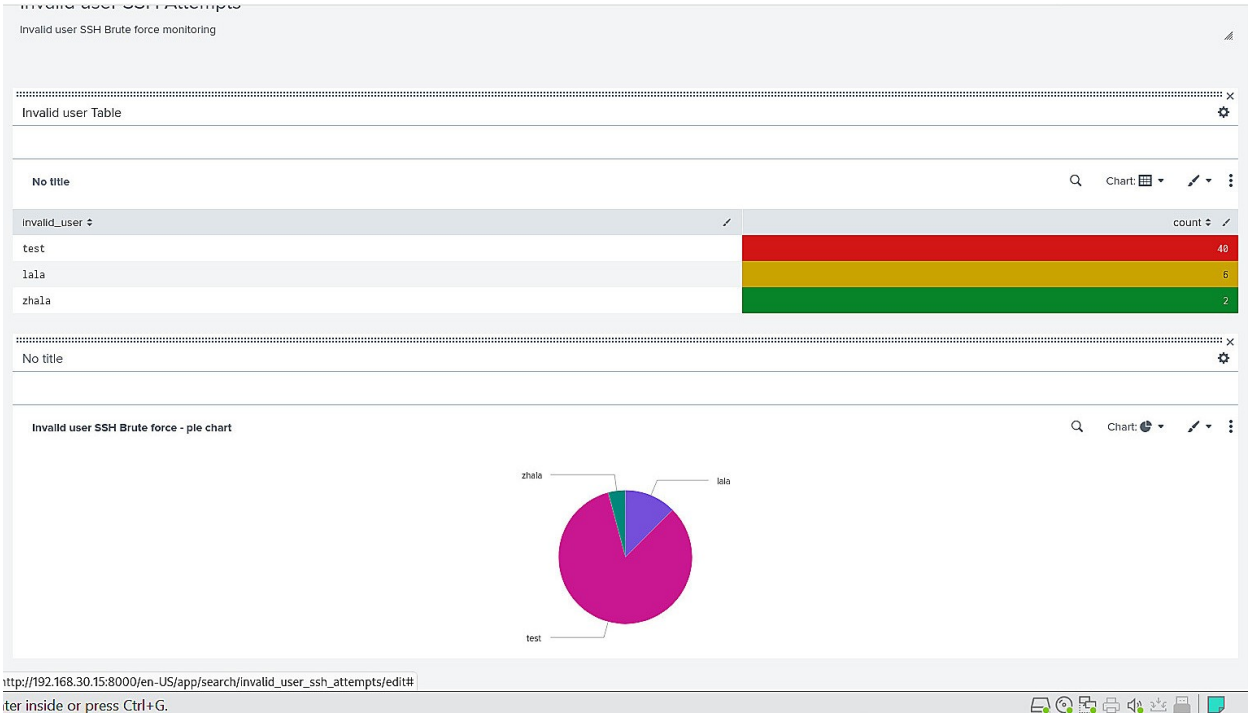


Figure 6. Dashboard panels showing failed SSH login patterns.

7. Mobile access with Splunk Secure Gateway

The project also explored Splunk Secure Gateway and the Splunk Mobile application for viewing dashboards on an approved device. This included enabling the gateway, registering the mobile client and verifying that the SOC dashboard could be viewed remotely. No device registration codes, user identifiers or account screenshots are reproduced here.

8. Email notifications

For proactive alerting, I configured SMTP-based email notifications in Splunk and created an alert linked to suspicious authentication activity. The rule specified a search, trigger criteria and a notification action. I tested the end-to-end mechanism and reviewed the resulting message.

Credentials, app passwords, personal email addresses and provider-specific settings are intentionally excluded from this public portfolio. The reusable lesson is the event-to-search-to-alert workflow, not the private values used by the test account.

9. Telegram notifications

I also worked with the Telegram Alert Action integration. A bot-based notification workflow sent selected Splunk alert information to a designated test destination. Alert content could use search-result fields, such as source IP and affected username, to provide context.

For privacy and security, this public report does not include bot tokens, chat IDs, personal conversations or screenshots of the messaging application.

Results and lessons learned

The result was a working lab workflow from endpoint log collection to central analysis, visualization and notification. It reinforced the practical importance of normalising fields, checking the reliability of extracted values and restricting cross-zone SIEM access to the required ports and hosts. Dashboards and alerts were based on the actual collected events, not on static mock data.

Scope: This is a portfolio account of the documented training exercise, not a claim of 24/7 production SOC operations.

Wazuh: Endpoint Monitoring, Threat Intelligence and Active Response

Project: Enterprise Security Infrastructure and SOC Monitoring

Area: Blue Team / Endpoint Detection and Response

Environment: Wazuh on Red Hat Enterprise Linux, Windows and Linux agents, Palo Alto NGFW

Objective and architecture

The Wazuh part of the project focused on endpoint visibility, configuration assessment, suspicious login activity, monitored file changes and automated defensive actions. An all-in-one Wazuh deployment was installed in the Management zone with three core components:

- **Wazuh Indexer:** stores and indexes incoming security events.
- **Wazuh Manager:** receives agent data and applies decoders, rules and active-response logic.
- **Wazuh Dashboard:** provides the interface for agent administration, investigation and visualisation.

The objective was to make endpoint events observable centrally and connect selected detections to alerting or response actions. All activity was carried out in a controlled virtual lab.

1. Installing the central Wazuh platform

22. Prepared the RHEL virtual machine and installed the all-in-one Wazuh stack using the documented installation workflow.
23. Confirmed availability of the Manager, Indexer and Dashboard components.
24. Configured the host firewall to support agent connections and enrolment.
25. Used the Wazuh console to organise endpoints into groups for Linux and Windows systems.

Separate groups allowed operating-system-specific policies and configurations to be managed more systematically.

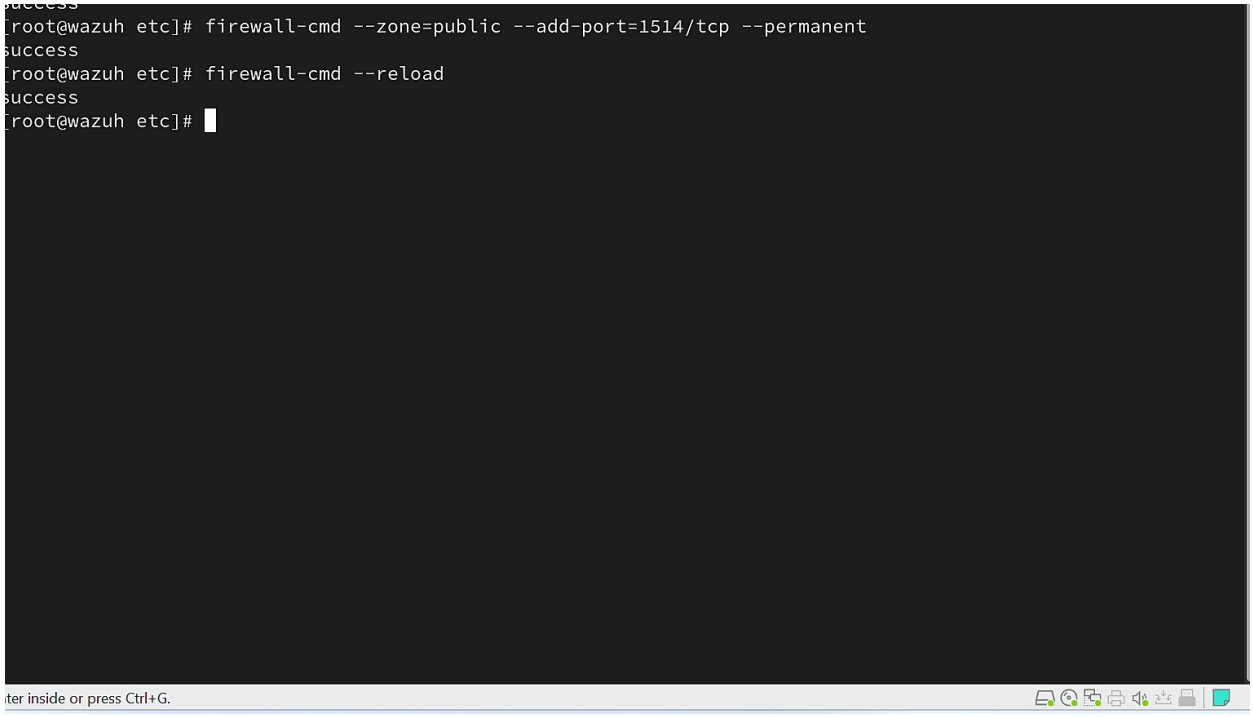


Figure 1. Host firewall configuration for Wazuh agent communication.

2. Restricting Wazuh traffic with Palo Alto NGFW

The Wazuh server was placed in the Management zone rather than exposed to every internal system without limits. A dedicated Palo Alto rule allowed endpoints to reach the specified Wazuh server on the required agent communication and enrolment ports:

Purpose	Protocol and port
Agent communication	TCP 1514
Agent enrolment	TCP 1515

The NGFW policy restricted the destination to the Wazuh host and limited the services to these defined ports. This was separate from access to the Wazuh administrative interface.

3. Deploying agents on Linux and Windows endpoints

For Linux systems, I generated an agent deployment command in the Wazuh console, selected the relevant package type and architecture, and set the Manager endpoint and assignment group. After installation, the service was enabled and started with systemd.

The project also included Windows machines, including servers and clients. I checked the registered agents from the central dashboard, compared their operating system information and confirmed the status of the monitored hosts. The agent view made it possible to distinguish connected hosts from inactive or unregistered ones.

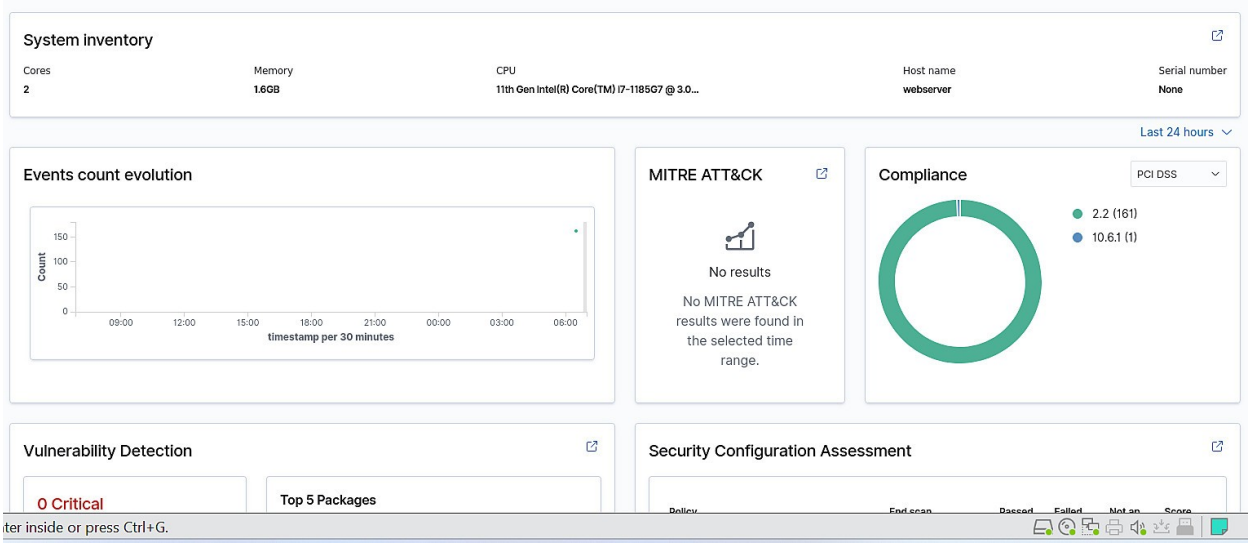


Figure 2. Registered endpoint and security-monitoring overview.

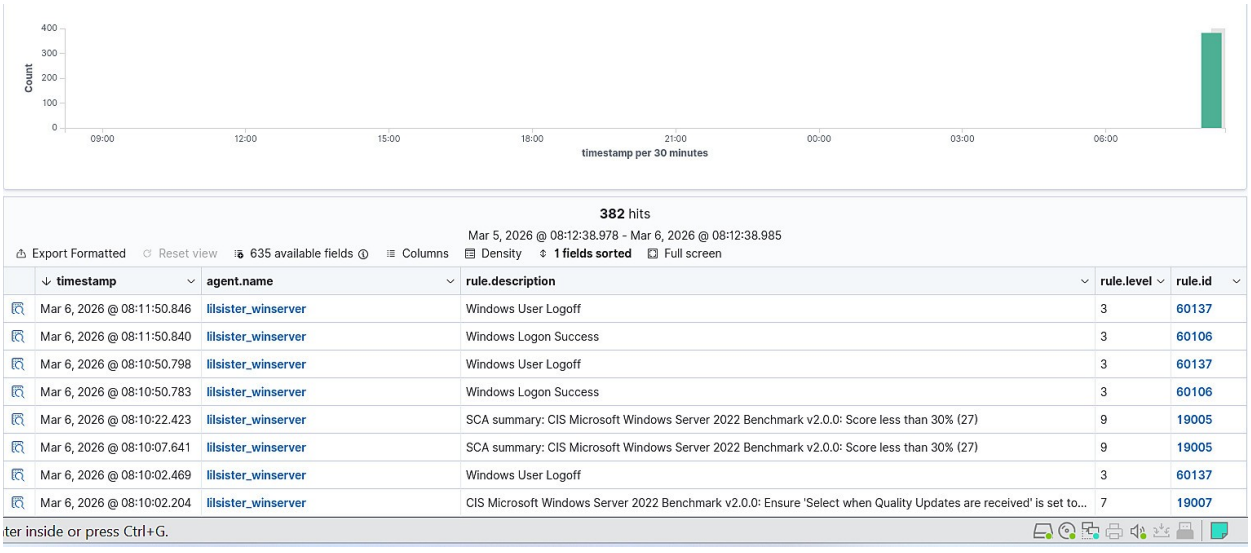


Figure 3. Endpoint events received by the Wazuh platform.

4. Security Configuration Assessment (SCA)

I used the built-in Security Configuration Assessment feature to examine security control results on monitored endpoints. The SCA view provided an overview of successful and failed configuration checks and surfaced areas requiring attention.

The assessment was used as a practical hardening reference: identify a failed check, understand which configuration it refers to, consider the effect of a change and reassess. A compliance score is useful for prioritisation, but it is not proof that a system is free from vulnerabilities.

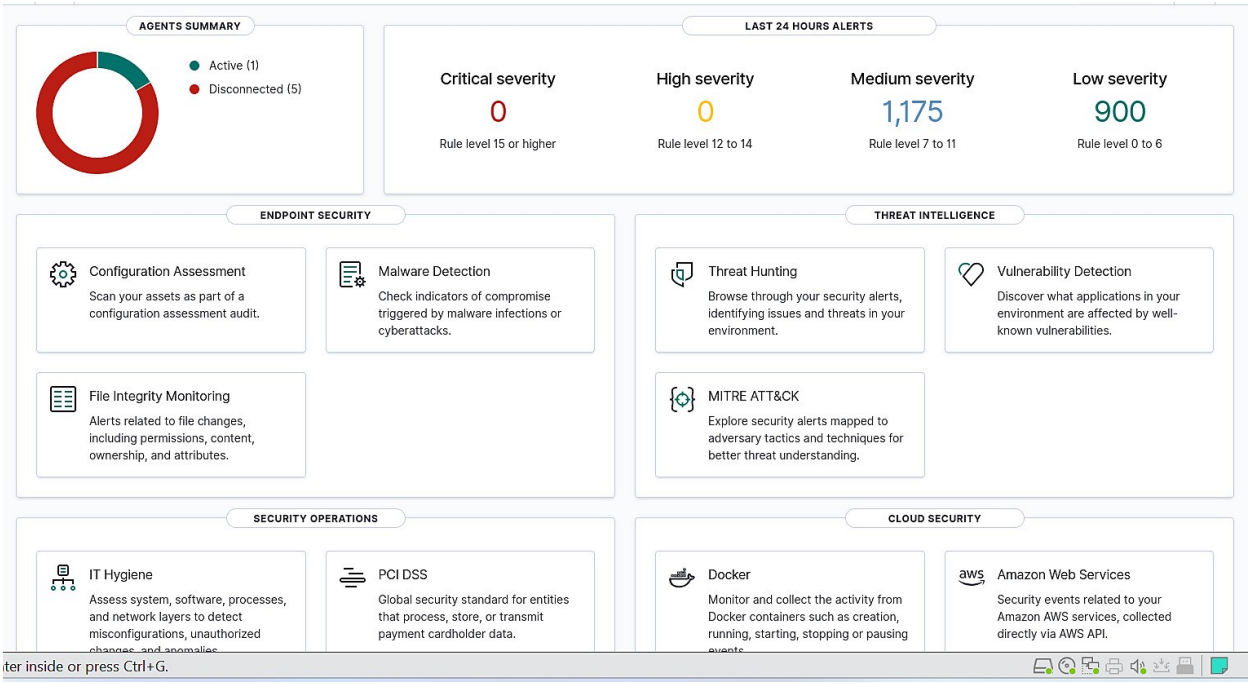


Figure 4. Security Configuration Assessment results.

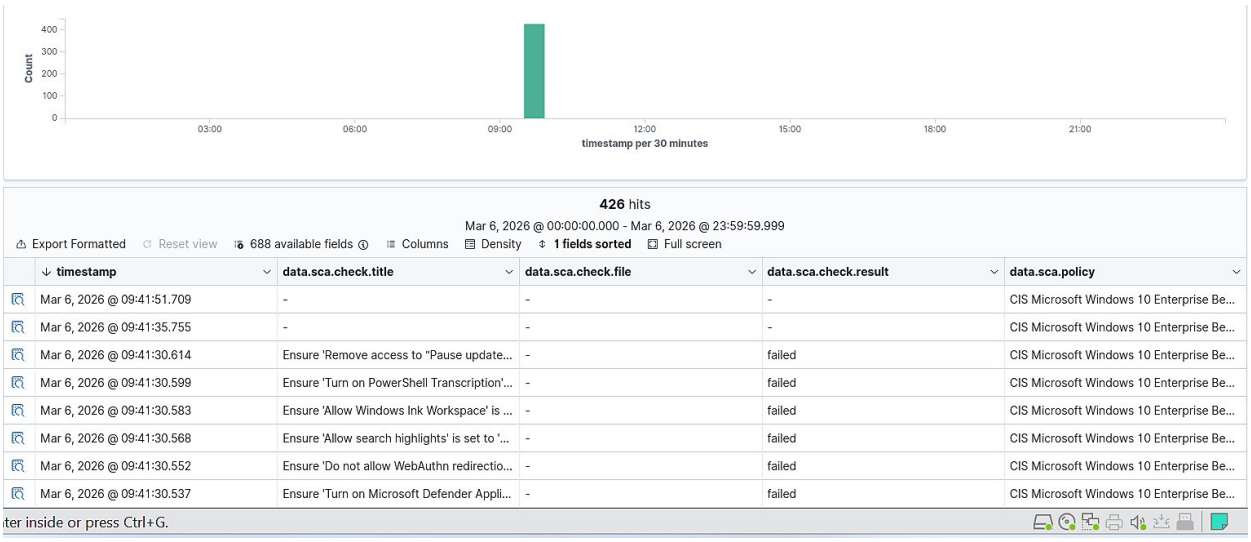


Figure 5. Detailed assessment events and checks.

5. SSH monitoring and a custom dashboard

The project included a custom dashboard dedicated to SSH authentication failures. In Discover, I filtered security events related to failed logins, including relevant Wazuh SSH/sshd rule messages. The data was then used to build:

- A table of source IP addresses, targeted usernames and counts of failed attempts.
- A pie chart to display how authentication failures were distributed across targeted accounts.
- A dashboard combining the visualizations with a time filter.

This approach made repeated attempts easier to locate without manually reading each raw log entry. The dashboard supported investigation; it was not an automatic determination that every failed login was malicious.

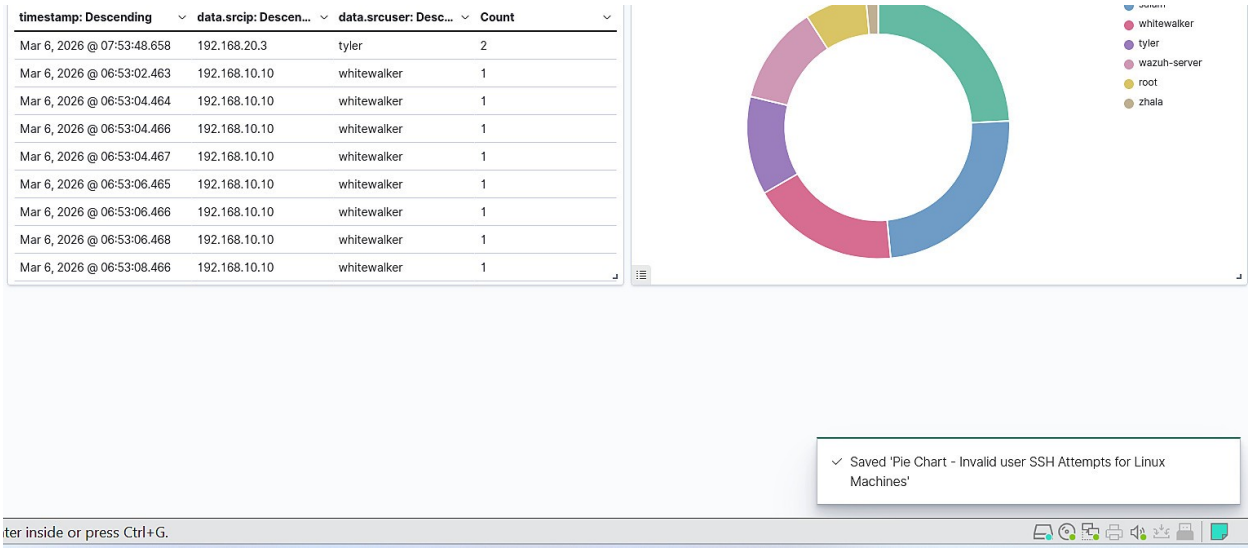


Figure 6. Custom dashboard for SSH failure analysis.

6. File Integrity Monitoring (FIM) and VirusTotal integration

Wazuh File Integrity Monitoring was configured to watch selected directories and report file creation or modification. The monitoring workflow generated file hashes that could be used for threat intelligence lookups. The project then integrated VirusTotal through Wazuh Manager to evaluate the reputation of those hashes.

The sequence was:

26. Configure watched directories through the agent's <syscheck> settings.
27. Enable appropriate file metadata and hash collection.
28. Configure the VirusTotal integration in the Manager's <integration> section.
29. Generate a controlled test event and check the resulting alert fields in Wazuh.

The external reputation lookup used file **hashes**, not the contents of the entire file. The private VirusTotal API key and identifying account details have intentionally been omitted from this public version.

7. Active Response: file handling

The exercise explored automatic handling of a file flagged as malicious by the threat intelligence workflow. An Active Response rule was linked to a selected trigger so that a local endpoint command or script could be executed when that condition occurred.

The design included a response command, an event-based trigger and a chosen execution location. This is useful for understanding the path from detection to containment, but response scripts need to be tested carefully: a false positive could remove or isolate a legitimate file. This report documents the training configuration without publishing operational credentials, full script secrets or personal paths.

8. Active Response: temporary IP blocking

A second Active Response workflow used Wazuh's firewall-drop command to block traffic from an address associated with selected high-severity or repeated-authentication events. The response was configured with a time limit rather than as an indefinite block, reducing the risk of permanent lockout when conditions were misclassified.

In the reported exercise, the process included identifying a rule or severity trigger, connecting it to the firewall-drop command, selecting local agent execution and restarting the necessary agent service. Resulting events and firewall state were reviewed to check whether the desired response was applied.

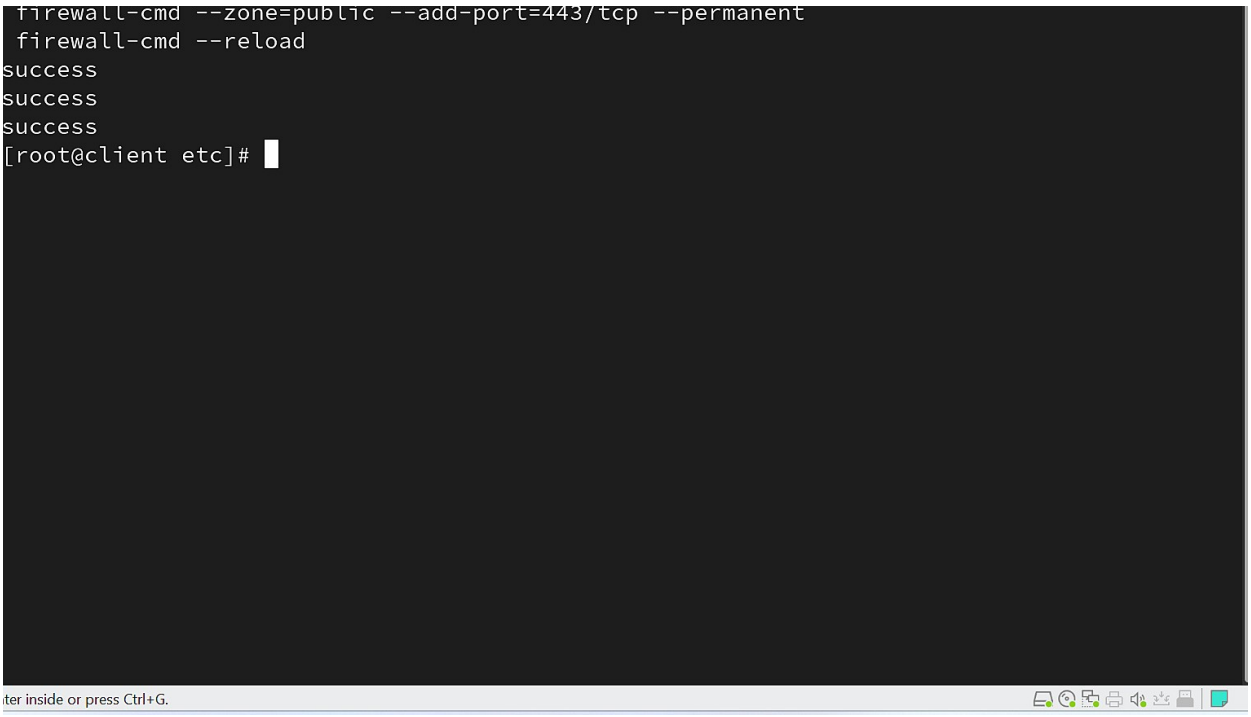


Figure 9. Endpoint firewall commands associated with the response exercise.

9. Email notifications

SMTP email notifications were configured to deliver selected higher-severity events instead of sending every low-level log entry. The configuration considered the notification destination, a severity threshold and contextual fields such as event time, affected endpoint and relevant indicator. This helped illustrate the tradeoff between prompt notification and unnecessary alert volume.

No email address, SMTP credentials or real account information is reproduced in this portfolio.

10. Telegram notifications

The project also explored sending Wazuh notifications through Telegram using a custom integration and webhook/script workflow. Selected events were eligible for forwarding based on severity or response conditions. This was a learning exercise in integrating monitoring platforms with external notification systems.

Bot tokens, chat identifiers, private messages and integration secrets are intentionally excluded. The presence of this capability in a lab should not be confused with an audited production incident-notification service.

Results and lessons learned

The Wazuh deployment connected Windows and Linux endpoint telemetry to centralised event analysis. It supported configuration assessment, SSH anomaly monitoring, FIM, VirusTotal reputation enrichment and rule-based responses. The project was particularly useful for understanding the difference between **detecting** an event, **enriching** it with external data, **notifying** an analyst and **acting** on it through Active Response. Each step requires its own validation, and automated actions must be designed with false-positive risk in mind.

Scope: Controlled academic lab. The report describes the implemented and documented configuration and tests; it does not claim zero-day detection, guaranteed malware prevention or production-grade reliability.